
Homework #3 — Trading and Portfolio Optimization

Table of Contents

Data loading	1
1 — Linear Price Predictor	1
2a — Logistic Regression	4
2b — Portfolio Strategies	7

Statistical Signal Processing and Learning, AA2025-2026

A portfolio of 10 NASDAQ technology stocks is traded over 2018. The first semester (Jan-Jun, ~124 trading days) trains the models; the second semester (Jul-Dec) evaluates them out of sample. Starting capital: 10.000 EUR.

Data loading

```
clc; clear; close all;
load("DataStocks/Hw3.mat");

open_prices = table();
close_prices = table();
total_volumes = table();

for i = 1:length(Stock)
    ticker = strtrim(Stock(i,:));
    td = eval(ticker);
    open_prices.(ticker) = td(2:end,1);
    close_prices.(ticker) = td(2:end,4);
    total_volumes.(ticker) = td(2:end,5);
end

T_days = height(open_prices);
n_stocks = width(open_prices);
tickers = open_prices.Properties.VariableNames;
P_o = table2array(open_prices);
P_c = table2array(close_prices);
Vol = table2array(total_volumes);

N_train = 124;
idx_tr = 1:N_train;
idx_te = (N_train+1):T_days;
C0 = 10000;
```

1 — Linear Price Predictor

For each stock i , the closing price is estimated at the opening of day t :

$$\hat{P}_i(t+1|t) = \mathbf{b}_i^T \mathbf{x}_i(t)$$

The feature vector uses the current opening price, the four most recent closing prices, and the previous day's volume:

$$\mathbf{x}_k(t) = [1, P_o^k(t), P_c^k(t-1), \dots, P_c^k(t-4), V_k(t-1)/10^6]^T$$

Coefficients $\mathbf{b}_k$ are estimated by ordinary least squares on the training set. At each test day the entire capital is placed on the single stock with the highest predicted intraday gain $(\hat{P}_k - P_o^k)P_c^k$.

```
L = 5;
B = zeros(L+2, n_stocks);

for k = 1:n_stocks
    valid = idx_tr(idx_tr > L);
    Xtr = zeros(length(valid), L+2);
    ytr = zeros(length(valid), 1);
    for ii = 1:length(valid)
        t = valid(ii);
        Xtr(ii,:) = [1, P_o(t,k), P_c(t-1,k), P_c(t-2,k), ...
                    P_c(t-3,k), P_c(t-4,k), Vol(t-1,k)/1e6];
        ytr(ii) = P_c(t,k);
    end
    B(:,k) = (Xtr'*Xtr) \ (Xtr'*ytr);
end

% Trading simulation
C = zeros(length(idx_te)+1, 1); C(1) = C0;
pred_g = zeros(length(idx_te), n_stocks);
act_g = zeros(length(idx_te), n_stocks);

for ii = 1:length(idx_te)
    t = idx_te(ii);
    P_hat = zeros(n_stocks,1);
    for k = 1:n_stocks
        x_t = [1, P_o(t,k), P_c(t-1,k), P_c(t-2,k), ...
              P_c(t-3,k), P_c(t-4,k), Vol(t-1,k)/1e6]';
        P_hat(k) = B(:,k)' * x_t;
    end
    pred_g(ii,:) = (P_hat' - P_o(t,:)) ./ P_o(t,:);
    act_g(ii,:) = (P_c(t,:) - P_o(t,:)) ./ P_o(t,:);
    [bg, bk] = max(pred_g(ii,:));
    if bg > 0
        C(ii+1) = C(ii) * P_c(t,bk) / P_o(t,bk);
    else
        C(ii+1) = C(ii);
    end
end
g_t = diff(C) ./ C(1:end-1);

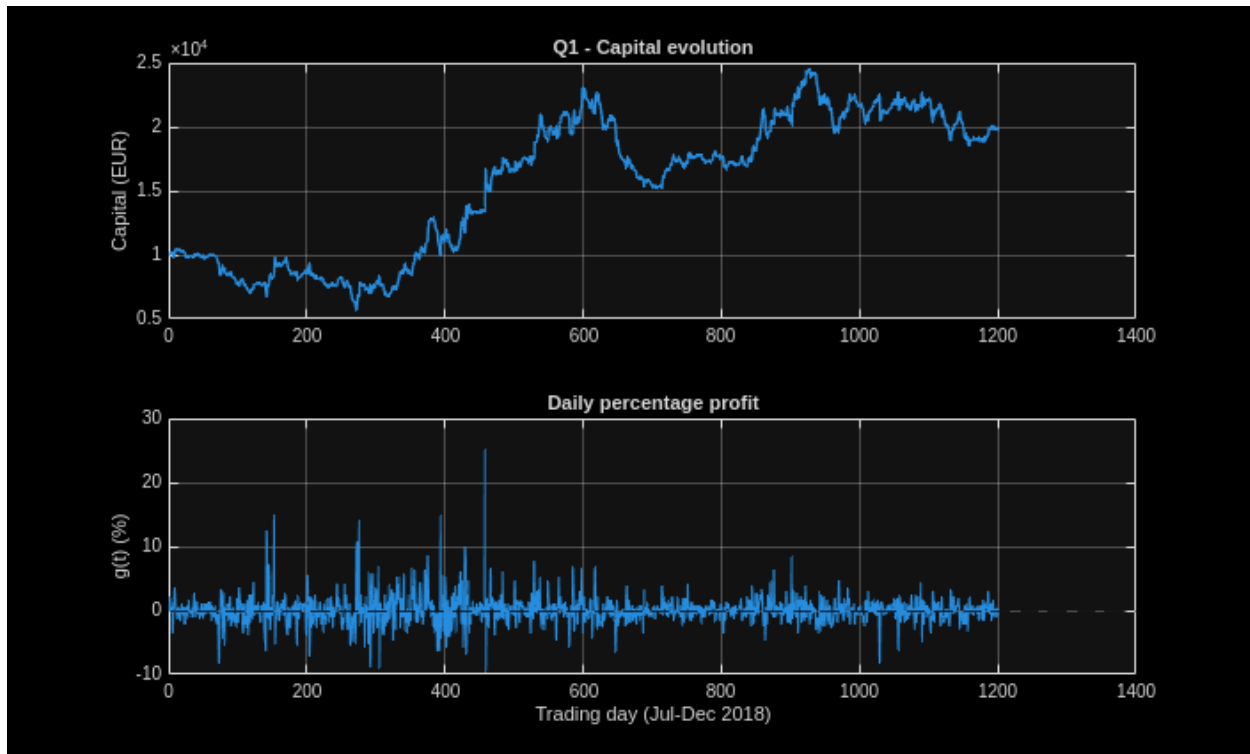
fprintf('Q1 - Final capital: %.2f EUR  (+%.1f%%)\n', C(end), 100*(C(end)-C0)/C0);
```

Q1 - Final capital: 19763.31 EUR (+97.6%)

Capital evolution $C(t)$ and daily percentage profit $g(t) = \Delta C(t)/C(t)$.

```
figure;
subplot(2,1,1);
plot(1:length(C), C, 'LineWidth', 1.5);
ylabel('Capital (EUR)'); title('Q1 - Capital evolution'); grid on;

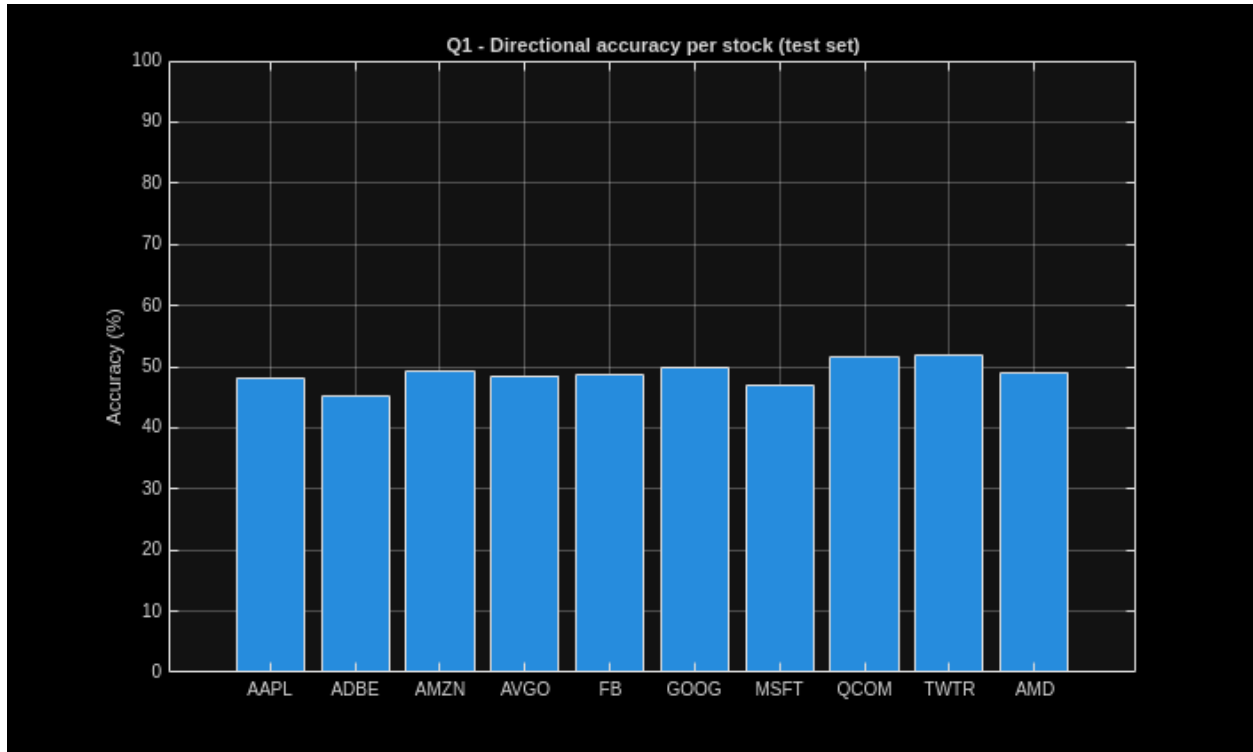
subplot(2,1,2);
plot(1:length(g_t), g_t*100, 'LineWidth', 1.2); yline(0, 'k--');
xlabel('Trading day (Jul-Dec 2018)');
ylabel('g(t) (%)'); title('Daily percentage profit'); grid on;
```



Directional prediction accuracy: fraction of test days where the sign of the predicted gain matches the actual intraday move.

```
correct1 = zeros(1, n_stocks);
for ii = 1:length(idx_te)
    correct1 = correct1 + ((act_g(ii,:) > 0) == (pred_g(ii,:) > 0));
end

figure;
bar(correct1 / length(idx_te) * 100);
xticks(1:n_stocks); xticklabels(tickers);
ylabel('Accuracy (%)');
title('Q1 - Directional accuracy per stock (test set)');
ylim([0 100]); grid on;
```



2a — Logistic Regression

The binary label $y_k(t) \in \{0, 1\}$ marks up-days: $y_k(t) = 1$ iff $P_k^u(t) > P_k^d(t)$. Buy probability is modelled by:

$$h_{b_k}(x_k(t)) = \frac{1}{1 + e^{-b_k^T x_k(t)}}$$

with the same feature vector as Q1. Parameters b_k are found by gradient descent on the binary cross-entropy (2000 steps, step size $\alpha = 0.1/N$). All features are standardised before training.

```
sigmoid = @(z) 1 ./ (1+exp(-z));
B_log   = zeros(L+2, n_stocks);
mu_all  = zeros(n_stocks, L+1);
sig_all = zeros(n_stocks, L+1);
acc_tr  = zeros(1, n_stocks);
acc_te  = zeros(1, n_stocks);
h_tr_all = cell(n_stocks,1);
y_tr_all = cell(n_stocks,1);

for k = 1:n_stocks
    valid_tr = idx_tr(idx_tr > L);
    N_tr = length(valid_tr);
    Xtr = zeros(N_tr, L+2); ytr = zeros(N_tr,1);
    for ii = 1:N_tr
        t = valid_tr(ii);
        Xtr(ii,:) = [1, P_o(t,k), P_c(t-1,k), P_c(t-2,k), ...
                    P_c(t-3,k), P_c(t-4,k), Vol(t-1,k)/1e6];
        ytr(ii) = P_c(t,k) > P_o(t,k);
    end
end
```

```

mu_x = mean(Xtr(:,2:end));
sig_x = std(Xtr(:,2:end));
Xtr_n = [ones(N_tr,1), (Xtr(:,2:end)-mu_x)./sig_x];
b = zeros(L+2,1);
for it = 1:2000
    hh = sigmoid(Xtr_n*b);
    b = b - (0.1/N_tr) * (Xtr_n'*(hh-ytr));
end
B_log(:,k) = b;
mu_all(k,:) = mu_x;
sig_all(k,:) = sig_x;
h_tr_all{k} = sigmoid(Xtr_n*b);
y_tr_all{k} = ytr;
acc_tr(k) = mean((h_tr_all{k} > 0.5) == ytr) * 100;

valid_te = idx_te(idx_te > L); N_te = length(valid_te);
Xte = zeros(N_te,L+2); yte = zeros(N_te,1);
for ii = 1:N_te
    t = valid_te(ii);
    Xte(ii,:) = [1, P_o(t,k), P_c(t-1,k), P_c(t-2,k), ...
                P_c(t-3,k), P_c(t-4,k), Vol(t-1,k)/1e6];
    yte(ii) = P_c(t,k) > P_o(t,k);
end
Xte_n = [ones(N_te,1), (Xte(:,2:end)-mu_x)./sig_x];
acc_te(k) = mean((sigmoid(Xte_n*b) > 0.5) == yte) * 100;
end

fprintf('\n%-6s  Train  Test\n','Stock');
for k = 1:n_stocks
    fprintf('%-6s  %5.1f%%  %5.1f%%\n', tickers{k}, acc_tr(k), acc_te(k));
end

```

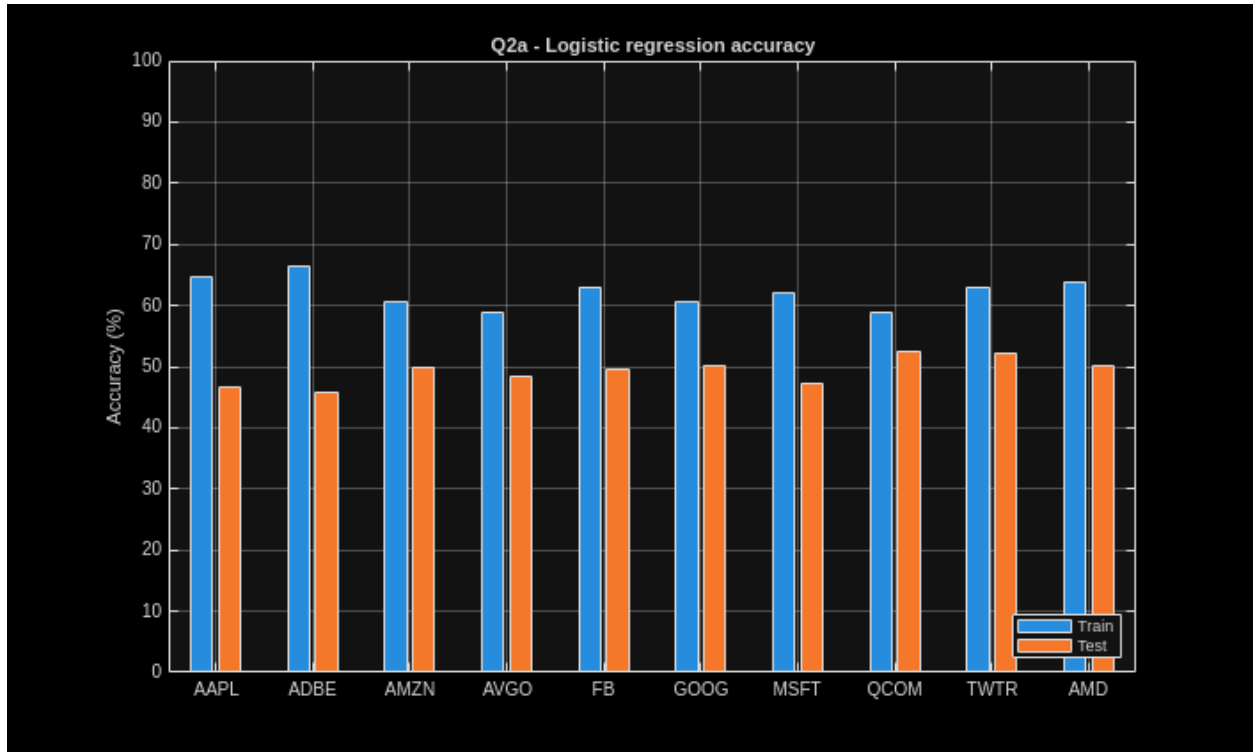
<i>Stock</i>	<i>Train</i>	<i>Test</i>
AAPL	64.7%	46.5%
ADBE	66.4%	45.8%
AMZN	60.5%	50.0%
AVGO	58.8%	48.3%
FB	63.0%	49.5%
GOOG	60.5%	50.2%
MSFT	62.2%	47.2%
QCOM	58.8%	52.4%
TWTR	63.0%	52.2%
AMD	63.9%	50.1%

Training accuracy reaches 60-68%; test accuracy drops to near 50%. The model captures in-sample patterns but stock direction is essentially unpredictable out of sample, especially in the volatile second half of 2018.

```

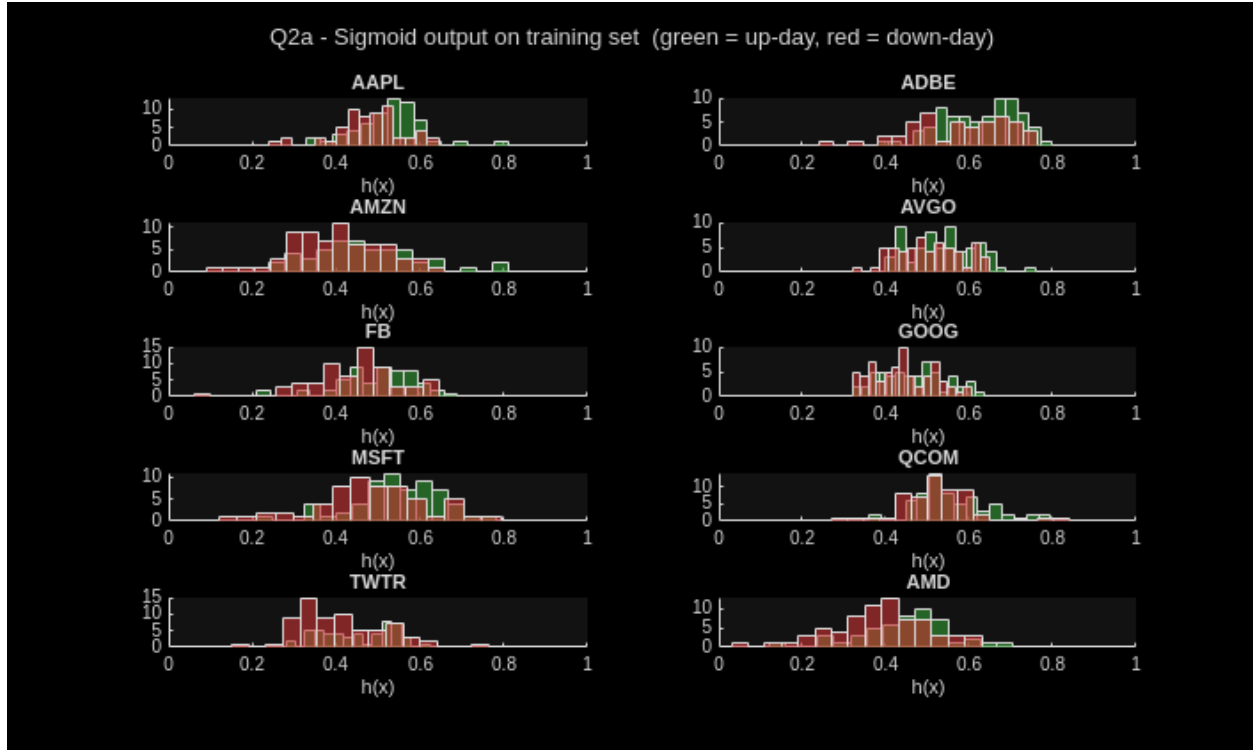
figure;
bar([acc_tr; acc_te]');
xticks(1:n_stocks); xticklabels(tickers);
ylabel('Accuracy (%)'); title('Q2a - Logistic regression accuracy');
legend({'Train', 'Test'}, 'Location', 'southeast'); ylim([0 100]); grid on;

```



Sigmoid output distributions on the training set. A well-calibrated model would shift the green (up-day) mass towards 1 and the red (down-day) mass towards 0. The significant overlap confirms limited separability.

```
figure;  
for k = 1:n_stocks  
    subplot(5,2,k); hold on;  
    histogram(h_tr_all{k}(y_tr_all{k}==1), 15, 'FaceColor',[0.2 0.6 0.2]);  
    histogram(h_tr_all{k}(y_tr_all{k}==0), 15, 'FaceColor',[0.8 0.2 0.2]);  
    title(tickers{k}); xlabel('h(x)'); xlim([0 1]); hold off;  
end  
sgtitle('Q2a - Sigmoid output on training set (green = up-day, red = down-day)');
```



2b — Portfolio Strategies

Three allocation rules are tested on the test period using the buy probabilities $h_k(t)$ produced by the logistic models:

- **S1 - winner-takes-all:** all capital on $\arg \max_k h_k(t)$ provided $\max_k h_k > 0.5$; otherwise hold.
- **S2 - proportional:** split capital proportional to h_k among all stocks with $h_k > 0.5$; hold if none qualify.
- **S3 - high-confidence:** same as S2 with the threshold raised to $h_k > 0.6$, keeping only the most confident predictions.

```
N_te_s = length(idx_te);
C1 = zeros(N_te_s+1,1); C1(1) = C0;
C2 = zeros(N_te_s+1,1); C2(1) = C0;
C3 = zeros(N_te_s+1,1); C3(1) = C0;

for ii = 1:N_te_s
    t = idx_te(ii);
    h = zeros(n_stocks,1);
    for k = 1:n_stocks
        x_t = [1, (P_o(t,k) - mu_all(k,1))/sig_all(k,1), ...
                (P_c(t-1,k) - mu_all(k,2))/sig_all(k,2), ...
                (P_c(t-2,k) - mu_all(k,3))/sig_all(k,3), ...
                (P_c(t-3,k) - mu_all(k,4))/sig_all(k,4), ...
                (P_c(t-4,k) - mu_all(k,5))/sig_all(k,5), ...
                (Vol(t-1,k)/1e6 - mu_all(k,6))/sig_all(k,6)]';
        h(k) = sigmoid(B_log(:,k)' * x_t);
    end
    ret = P_c(t,:) ./ P_o(t,:);
```

```

[bh, bk] = max(h);
if bh > 0.5; C1(ii+1)=C1(ii)*ret(bk); else; C1(ii+1)=C1(ii); end

m2 = h > 0.5;
if any(m2); w=h.*m2/sum(h.*m2); C2(ii+1)=C2(ii)*(w'*ret); else;
C2(ii+1)=C2(ii); end

m3 = h > 0.6;
if any(m3); w=h.*m3/sum(h.*m3); C3(ii+1)=C3(ii)*(w'*ret); else;
C3(ii+1)=C3(ii); end
end

fprintf('\nQ2b - Final capitals:\n');
fprintf(' S1 winner-takes-all %.2f EUR (%+.1f%%)\n', C1(end), 100*(C1(end)-
C0)/C0);
fprintf(' S2 proportional %.2f EUR (%+.1f%%)\n', C2(end), 100*(C2(end)-
C0)/C0);
fprintf(' S3 high-confidence %.2f EUR (%+.1f%%)\n', C3(end), 100*(C3(end)-
C0)/C0);

```

Q2b - Final capitals:

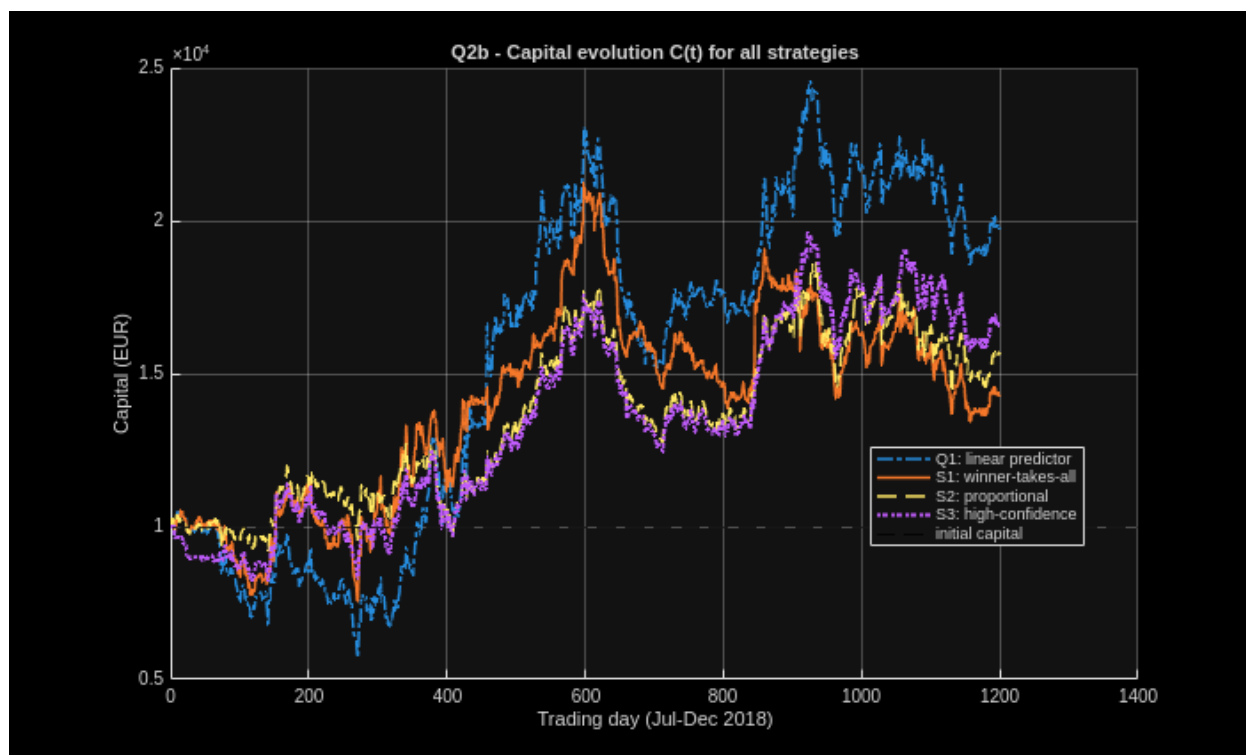
S1 winner-takes-all	14289.03 EUR	(+42.9%)
S2 proportional	15678.32 EUR	(+56.8%)
S3 high-confidence	16616.66 EUR	(+66.2%)

Capital evolution $C(t)$ from Jul-Dec 2018 for all four strategies. The Q1 linear predictor outperforms all logistic strategies due to its aggressive single-stock allocation and good performance on a few large up-moves early in the test period.

```

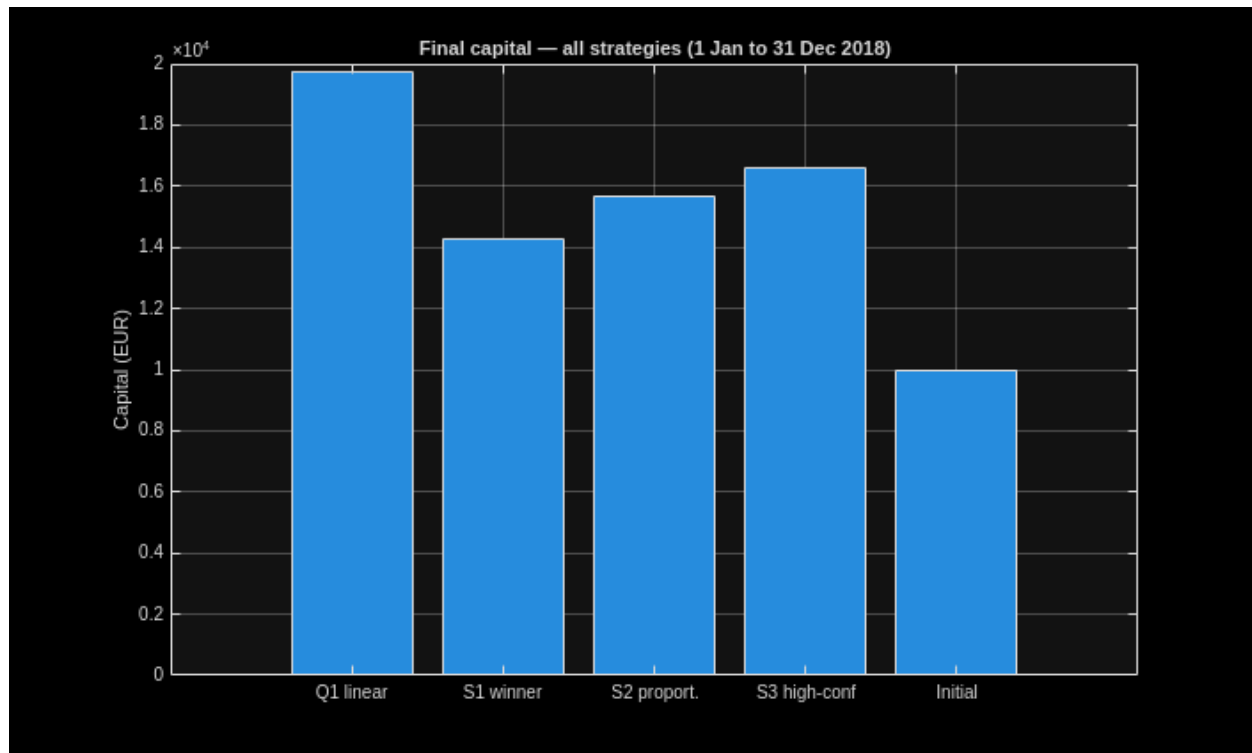
t_ax = 0:N_te_s;
figure;
hold on;
plot(t_ax, C, '-.', 'LineWidth', 1.5, 'DisplayName', 'Q1: linear predictor');
plot(t_ax, C1, '-', 'LineWidth', 1.5, 'DisplayName', 'S1: winner-takes-all');
plot(t_ax, C2, '--', 'LineWidth', 1.5, 'DisplayName', 'S2: proportional');
plot(t_ax, C3, ':', 'LineWidth', 2, 'DisplayName', 'S3: high-confidence');
yline(C0, 'k--', 'LineWidth', 1, 'DisplayName', 'initial capital');
xlabel('Trading day (Jul-Dec 2018)'); ylabel('Capital (EUR)');
title('Q2b - Capital evolution C(t) for all strategies');
legend('Location', 'best'); grid on; hold off;

```

Summary of final capitals. Strategies differ substantially in risk profile: S1 concentrates risk (like Q1) while S2/S3 spread it across multiple stocks, trading peak gains for lower volatility.

```
figure;
bar([C(end), C1(end), C2(end), C3(end), C0]);
xticklabels({'Q1 linear', 'S1 winner', 'S2 proport.', 'S3 high-conf', 'Initial'});
ylabel('Capital (EUR)');
title('Final capital – all strategies (1 Jan to 31 Dec 2018)');
grid on;
```



Published with MATLAB® R2025b